

AULA 1

Agentes de IA

Sistemas Multiagentes com IA · Aula 1 · SDK openai-agents. Setas ← → (ou Espaço).

Tecla F: tela cheia.

Apresentação

Francisco Molina Jr. — +20 anos em desenvolvimento de software e ciência de dados (PHP/Laravel, Python, Java).

- Assessor na Enap — roadmap dos produtos digitais educacionais da Escola Virtual de Governo (EV.G)
- Professor no UniCEUB — Desenvolvimento de Sistemas, Estrutura de Dados, Python, Banco de Dados, Sistemas Operacionais
- MBA em Data Science e IA Aplicada — Enap
- Ministrou o curso IA para Devs — TRF1
- LinkedIn: [linkedin.com/in/francisco-molina-jr](https://www.linkedin.com/in/francisco-molina-jr)

Sistemas em produção, com IA

centraldanatacao.com.br

bigball.com.br

lcsaudeintegrativa.com.br

liderecapitalestrategico.com.br

circuitodenatacaoinfantil.com.br

financeiro.skinflow.com.br

maisluzaroma.com.br

radar.liderecapitalestrategico.com.br

Ementa — Sistemas Multiagentes com IA

50h · 10 encontros remotos síncronos de 5h · MBA em BI & Data Science (Ibmec) · ênfase em segurança pública.

- 1 · Fundamentos de agentes — ambiente, estado, memória, autonomia; agente × chatbot; 1º agente de extração de ocorrências (hoje — em 2 dias)
- 2 · Comunicação agente-a-agente (A2A) — Request/Response, Pub/Sub, MQTT, gRPC, FIPA-ACL
- 3 · Model Context Protocol (MCP) e integração — tool calling, APIs, bancos, controle de acesso
- 4 · Engenharia de agent loops — ReAct, critérios de parada, retries, autonomia
- 5 · Padrões de orquestração — Supervisor, Broker, Blackboard, Pipeline, Debate
- 6 · Sistemas baseados em grafos — DAGs, estados compartilhados, LangGraph
- 7 · Agentes especializados e Human-in-the-Loop — papéis (Investigador, Jurídico, Analista), breakpoints
- 8 · Arquitetura distribuída e deploy — Docker, Kubernetes
- 9 · Auto-scaling, observabilidade e segurança — tracing, prompt injection, excessive agency, guardrails
- 10 · Projeto integrador — sistema multiagente fim a fim para um cenário de segurança pública

Avaliação e formato

- Avaliação da disciplina: o projeto integrador — um sistema multiagente fim a fim aplicado a um cenário de segurança pública, com desenvolvimento assistido e apresentação na Aula 10.
- Formato de cada aula: exposição dialogada + ~70% de laboratório hands-on em Python.
- Nesta turma: exemplos com o SDK openai-agents — backend OpenAI ou Ollama local (sem custo, nada sai da máquina).
- Exercícios: cada seção tem um desafio — o slide mostra só o enunciado; a solução é revelada depois de vocês tentarem.

Vamos montar a pasta do zero

Cinco passos. Ao final, qualquer exemplo da aula roda com um só comando.

- 1 — criar a pasta aula1
- 2 — criar o arquivo .env
- 3 — criar o arquivo requirements.txt
- 4 — entrar na pasta e configurar a venv
- 5 — implementar o provedor.py

Criar a pasta aula1

Tudo da aula vive numa pasta só. Escolha um lugar simples de achar depois.

```
mkdir aula1  
cd aula1
```

```
# caminho de exemplo:
```

```
# Windows:      C:\curso-ia\aula1
```

```
# Linux/macOS:  ~/curso-ia/aula1
```

Arquivo .env

Guarda o que muda por máquina — backend, modelo e chave — fora do código. Nunca vai para o Git.

```
# crie um arquivo chamado .env dentro de aula1/ e cole:

# backend: "openai" (padrão) ou "ollama"
PROVEDOR=openai

# modelo usado por TODOS os exemplos (nenhum passa model= no código)
OPENAI_DEFAULT_MODEL=gpt-4o-mini

# necessária só quando PROVEDOR=openai
OPENAI_API_KEY=coloque_sua_chave_aqui

# usada só quando PROVEDOR=ollama (este já é o valor padrão)
# OLLAMA_BASE_URL=http://localhost:11434/v1
```

Ollama e a variável PATH

Roda o modelo na sua máquina. Depois de instalar, o comando ollama só é reconhecido se ele estiver no PATH.

```
# 1) instalar – escolha UMA opção:
# a) baixar o instalador em https://ollama.com/download (Windows / macOS / Linux)
# b) linha de comando:
# Windows: winget install --id Ollama.Ollama -e
# Linux: curl -fsSL https://ollama.com/install.sh | sh

# 2) PATH – fazer o comando ollama ser reconhecido
# O instalador ADICIONA o Ollama ao PATH, mas terminais já abertos não veem.
# Windows (definitivo): feche TODOS os terminais / o VS Code e reabra
# Windows (só nesta sessão):
$env:Path += ";$env:LOCALAPPDATA\Programs\Ollama"
# conferir o PATH permanente do usuário:
[Environment]::GetEnvironmentVariable("Path", "User")
# Linux/macOS: reabra o shell (o binário fica em /usr/local/bin, já no PATH)

# 3) conferir e baixar o modelo
ollama --version
ollama pull llama3.1          # ~4.9 GB – baixar em casa
ollama run llama3.1 "diga ola" # teste

# 4) no .env: PROVIDOR=ollama | OPENAI_DEFAULT_MODEL=llama3.1
```

Arquivo requirements.txt

A lista de bibliotecas da aula. O comando `pip install -r` lê daqui.

```
# crie um arquivo requirements.txt dentro de aula1/ e cole:

openai-agents>=0.22.0,<0.23.0    # o SDK de agentes
python-dotenv>=1.0.1            # lê o arquivo .env
pydantic>=2.7                   # modelos de saída (seção 4)
requests>=2.31                  # chamadas HTTP das ferramentas (seção 2 em diante)
```

Entrar na pasta e criar a venv

A venv é uma caixa isolada: as bibliotecas ficam só nesta pasta, sem conflitar com o resto do sistema. Ative-a em cada terminal novo.

```
cd aula1                                # se ainda não estiver nela

python -m venv .venv                    # cria a caixa isolada (precisa de Python 3.10+)

# ativar a venv:
#   Windows PowerShell:   .venv\Scripts\Activate.ps1
#   Windows cmd:         .venv\Scripts\activate.bat
#   Linux / macOS:       source .venv/bin/activate

pip install -r requirements.txt          # instala tudo do requirements.txt
pip show openai-agents                   # confere: Version: 0.22.x

# Windows, erro "execução de scripts desabilitada":
#   Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Implementar o provedor.py

Liga o SDK ao backend escolhido no .env. Todo exemplo começa com `from provedor import configurar; configurar()` — e trocar OpenAI ↔ Ollama passa a ser só editar o .env.

```
# crie o arquivo provedor.py dentro de aula1/ :

import os
from dotenv import load_dotenv
from openai import AsyncOpenAI
from agents import (
    set_default_openai_api, set_default_openai_client,
    set_default_openai_key, set_tracing_disabled,
)

load_dotenv() # carrega o .env para as variáveis de ambiente

def configurar() -> str:
    """Aplica o provedor escolhido no .env. Devolve o nome dele."""
    provedor = os.getenv("PROVEDOR", "openai").strip().lower()

    if provedor == "ollama":
        base_url = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434/v1")
        # o Ollama exige api_key no construtor, mas ignora o valor
        cliente = AsyncOpenAI(base_url=base_url, api_key="ollama")
        set_default_openai_client(cliente) # manda tudo para o Ollama
        set_default_openai_api("chat_completions") # Ollama não tem a Responses API
        set_tracing_disabled(True) # não envia traces p/ a OpenAI
    else:
        set_default_openai_key(os.getenv("OPENAI_API_KEY"))

    return provedor
```

Outros modelos encaixam no mesmo cliente

O formato chat/completions virou padrão de fato. Trocar de backend costuma ser só trocar base_url + api_key no provedor.py.

```
# no provedor.py, um ramo como o do Ollama:  
cliente = AsyncOpenAI(base_url="https://api.groq.com/openai/v1",  
                      api_key=os.getenv("GROQ_API_KEY"))
```

O que não encaixa direto

- Ajuste pequeno — Azure OpenAI (cliente AsyncAzureOpenAI, api_version, nome do deployment) · Gemini (endpoint .../v1beta/openai/)
- API diferente — Claude nativo, AWS Bedrock, Vertex AI → precisa de um tradutor (LiteLLM; o SDK traz LitellmModel, ~100 provedores como se fossem OpenAI)
- Mesmo 'compatível' — function calling e output_type variam por modelo; a Responses API só a OpenAI e a Azure têm (por isso o provedor.py força chat_completions)

Um agente é um funcionário

Não é um chatbot

O agente pode decidir agir — chamar ferramentas, consultar sistemas, passar o caso para outro setor — e repetir isso em ciclo até ter a resposta. Não é só "gerar texto".

Cada seção adiciona uma capacidade ao mesmo funcionário

- 1 · responder — funcionário com manual
- 2 · usar ferramentas — dar a ele um sistema para consultar
- 3 · encaminhar (handoff) — mesa de triagem
- 4 · saída em formato fixo — preencher um formulário
- 5 · memória (sessão) — o prontuário do atendimento
- 6 · guardrails — detector de metais na entrada e revisão na saída
- 7 · streaming — ver o funcionário digitando
- 8 · tudo junto — o atendimento completo
- 9 · limites e produção — o que separa a demo do sistema real

SEÇÃO 1

O primeiro agente

responder — funcionário com um manual

EXEMPLOS DE APLICAÇÃO

- Agente de onboarding: recebe o novo servidor, explica os primeiros passos (SEI, e-mail institucional, crachá) sempre no mesmo roteiro e no mesmo tom.
- Dúvidas de RH: férias, licenças, progressão — responde citando a norma, sem improviso.
- FAQ ao cidadão: como tirar atestado de antecedentes, onde agendar — responde só o que está no script, sem inventar.

SAIBA MAIS

Um **Agent** é a peça básica do SDK: um modelo de linguagem + um conjunto de **instructions** (o "system prompt", que manda mais que o pedido do usuário) + um **Runner** que de fato executa o ciclo. Sozinho ele não faz nada — é só a definição.

SDK docs — Agents <https://openai.github.io/openai-agents-python/agents/>

O primeiro agente possível

Agent = LLM + instructions. É só uma definição; quem executa é o Runner. `run_sync` devolve um objeto rico (`RunResult`), não uma string.

```
from agents import Agent, Runner
from provedor import configurar
configurar()

agent = Agent(
    name="Meu Primeiro Agente",
    instructions=...,          # ← a "persona": o que ele SEMPRE responde
)

resultado = Runner.run_sync(agent, "O que vc faz?")
print(resultado)              # o objeto RunResult inteiro – o que tem aqui dentro?
print(resultado.____)         # ← só a resposta em texto
```

Dois hábitos para a aula toda

DESAFIO

Reescreva o 1.1 com dois hábitos que vão se repetir a aula toda: (1) usar `resultado.final_output` — só o texto, não o objeto; (2) pôr a execução dentro de uma função `main()`. Dê ao agente instruções abertas (um assistente de verdade) e observe que a resposta passa a depender da pergunta.

 Solução revelada em aula com a senha da turma

O mesmo agente, 100% local (Ollama)

Idêntico ao 1.1, mas rodando no Ollama — sem custo, nada sai da máquina. Só muda `model="llama3.1"` e `PROVEDOR=ollama` no `.env`.

```
import os
os.environ["PROVEDOR"] = _____ # ← forçar o backend local
os.environ.setdefault("OPENAI_DEFAULT_MODEL", "llama3.1")

from agents import Agent, Runner
from provedor import configurar
configurar() # o provedor.py aponta para localhost:11434

agent = Agent(
    name="Meu Primeiro Agente Ollama",
    instructions="Você é um agente de teste. Responda com 'Olá, mundo!'",
    model=_____, # ← modelo servido pelo Ollama
)
r = Runner.run_sync(agent, "O que vc faz?")
print("Agente:", r.last_agent.name) # a resposta obedece a instrução?
```

Ferramentas (tools)

usar ferramentas — dar a ele um sistema/telefone para consultar

EXEMPLOS DE APLICAÇÃO

- Onboarding com banco de dados: uma tool consulta a base institucional e o agente responde 'seu chefe imediato é X, seu setor é Y, você tem os treinamentos Z pendentes'.
- Consulta a sistema interno: tool no SEI / protocolo que devolve o andamento de um processo pelo número.
- Enriquecimento de dados: dado um CNPJ, CPF ou placa, a tool bate em bases cadastrais e o agente monta um resumo.
- Cálculo e validação: tool que valida um documento, calcula prazo processual ou converte datas — o modelo não faz conta, a função faz.

SAIBA MAIS

Tools (function calling) são funções Python comuns que o próprio modelo decide chamar durante a execução, para consultar um sistema, calcular algo ou buscar um dado que ele não sabe de cabeça. O decorator `@function_tool` gera o schema a partir da assinatura e da docstring da função.

SDK docs — Tools <https://openai.github.io/openai-agents-python/tools/>

A primeira ferramenta

@function_tool transforma uma função Python em ferramenta. O schema vem da assinatura + docstring — por isso a docstring não é enfeite, o modelo a lê para decidir quando e como usar.

```
from agents import Agent, Runner, function_tool

@function_tool # ← o que transforma isto numa ferramenta?
def consultar_clima(cidade: str) -> str:
    """Retorna o clima atual de uma cidade.

    Args:
        cidade: nome da cidade a consultar. # ← a docstring é LIDA pelo modelo
    """
    dados = {"São Paulo": "22°C, nublado", "Brasília": "28°C, sol"}
    return dados.get(cidade, "Cidade não encontrada.")

agente = Agent(
    name="Assistente de Clima",
    instructions="Use a ferramenta quando perguntarem sobre o tempo.",
    tools=[consultar_clima], # ← como o agente recebe a ferramenta?
)
```

Ferramenta com API real (Open-Meteo)

DESAFIO

Refaça o agente de clima do 2.1, mas com uma API real e gratuita (Open-Meteo). A ferramenta deve: (1) transformar o nome da cidade em coordenadas (endpoint de *geocoding*); (2) usar as coordenadas para pegar a temperatura atual (endpoint de *forecast*). Se a cidade não existir, `raise ValueError(...)` — o SDK entrega o erro ao modelo.

 Solução revelada em aula com a senha da turma

2.3 · FERRAMENTAS (TOOLS)

Desafio · câmbio: ferramenta com 2 argumentos

DESAFIO

Crie uma ferramenta que consulte a cotação entre duas moedas na Frankfurter (<https://api.frankfurter.app/latest>, dados do BCE, sem cadastro). A ferramenta recebe **dois** argumentos (`moeda_origem`, `moeda_destino`) e o modelo extrai os dois da frase (“dólar em reais” → USD, BRL). Monte um “Assistente Financeiro” e teste com “Quanto está o dólar hoje?”.

 Solução revelada em aula com a senha da turma

Desafio · CEP: retorno em texto estruturado

DESAFIO

Dado um CEP, consulte o endereço na ViaCEP (<https://viacep.com.br/ws/{cep}/json/>). Limpe o CEP (com ou sem hífen/ponto). Pegadinha: CEP inexistente **não** dá erro HTTP — vem `{"erro": "true"}`. Retorne logradouro, bairro, cidade e UF num texto; o modelo reescreve de forma natural.

 Solução revelada em aula com a senha da turma

2.5 · FERRAMENTAS (TOOLS)

Desafio · CNPJ + a 1ª conversa séria de LGPD

DESAFIO

Consulte um CNPJ na BrasilAPI (<https://brasilapi.com.br/api/cnpj/v1/{cnpj}>), dados públicos da Receita). Retorne razão social, situação cadastral, atividade, endereço e o quadro de sócios (qsa). Repare: os CPFs dos sócios já vêm mascarados pela fonte — a minimização acontece na origem.

 Solução revelada em aula com a senha da turma

Handoff (transferência entre agentes)

encaminhar — mesa de triagem que manda para o setor certo

EXEMPLOS DE APLICAÇÃO

- Central de atendimento ao cidadão: um triador separa denúncia, pedido de informação, reclamação e agendamento — cada um um agente especialista.
- Mesa de apoio à investigação: roteia entre especialista cadastral, de georreferenciamento e de telefonia, conforme a solicitação.
- Suporte de TI interno: separa 'acesso e senha', 'equipamento' e 'sistema X' e encaminha para a fila certa — sem if/else escrito por você.

SAIBA MAIS

Handoff é a transferência de uma conversa de um agente para outro: por baixo dos panos, cada agente disponível vira uma ferramenta (`transfer_to_<nome>`) que o agente de origem pode chamar. É o padrão para triagem.

SDK docs — Handoffs
<https://openai.github.io/openai-agents-python/handoffs/>

Triador + especialistas

O triador tem `handoffs=[...]` em vez de `tools=[...]`. Cada agente vira uma ferramenta `transfer_to_<name>`. A decisão é só linguagem natural — nenhum `if/else` de roteamento. `resultado.last_agent` diz quem respondeu.

```
agente_matematica = Agent(name="Especialista_Matematica",    # ← por que sem acento?
    instructions="Resolve matemática, com passo a passo.")
agente_historia = Agent(name="Especialista_Historia",
    instructions="Responde história de forma clara.")

triador = Agent(
    name="Triador",
    instructions="Encaminhe para o especialista certo. Não responda você mesmo.",
    _____=[agente_matematica, agente_historia],    # ← tools ou handoffs?
)

r = Runner.run_sync(triador, "Quem foi Pitágoras?")
print(r.____.name)                                     # ← quem realmente respondeu?
```

Observar e customizar a transferência

DESAFIO

A partir do 3.1, use `handoff(agente, on_handoff=callback)` no lugar do agente puro. O *callback* deve registrar um `[LOG]` no momento da transferência (auditoria). Faça o triador responder sozinho o que for trivial (“Bom dia!”) e só encaminhar as dúvidas de matemática/história.

 Solução revelada em aula com a senha da turma

Desafio · Central de Atendimento (handoff + tools)

DESAFIO

Central de Atendimento ao Cidadão: dois especialistas, cada um com sua ferramenta de API real — clima (Open-Meteo) e endereços (ViaCEP) — e um triador que só roteia. São duas decisões independentes do modelo. Teste com duas perguntas de assuntos diferentes e imprima quem respondeu.

 Solução revelada em aula com a senha da turma

Desafio · Apoio à Investigação (Nominatim)

DESAFIO

Central de Apoio à Investigação: um especialista cadastral (CNPJ/BrasilAPI) e um de localização (georreferenciamento via Nominatim, que converte endereço em coordenadas). Atenção: o Nominatim **exige** um header **User-Agent** identificando a aplicação — sem ele a requisição é bloqueada.

 Solução revelada em aula com a senha da turma

3.5 · HANDOFF (TRANSFERÊNCIA ENTRE AGENTES)

Desafio · o custo de um 3º especialista

DESAFIO

Partindo do 3.4, adicione um terceiro especialista, de telefonia, com uma ferramenta de DDD (<https://brasilapi.com.br/api/ddd/v1/{ddd}>) que retorne estado e municípios. São 3 passos: (1) a ferramenta, (2) o agente especialista, (3) uma linha em `handoffs` + uma frase nas instruções do triador. Nenhum `if/else`.

 Solução revelada em aula com a senha da turma

Saída estruturada (`output_type`)

saída em formato fixo — preencher um formulário, não escrever uma redação

EXEMPLOS DE APLICAÇÃO

- Registro de ocorrência: de um relato livre para um objeto {tipo, data, local, envolvidos[], resumo} pronto para gravar no sistema.
- Triagem de ouvidoria: cada manifestação vira {categoria, prioridade, urgente} e alimenta o painel automaticamente.
- Leitura de ofício ou e-mail: extrai {remetente, assunto, prazo, ação requerida} para uma fila de tarefas.
- Ficha de qualificação: de uma entrevista transcrita para campos fixos, revisados por um servidor antes de valer.

SAIBA MAIS

Saída estruturada (`output_type`) obriga o modelo a devolver dados num formato fixo, validado por um modelo Pydantic, em vez de texto livre — um contrato que o resto do sistema pode consumir sem parsing frágil de string.

SDK docs — Output types
<https://openai.github.io/openai-agents-python/agents/#output-types>

output_type com Pydantic

Declara um modelo Pydantic e o SDK obriga o modelo a devolver exatamente aquele formato, já convertido em objeto Python. `final_output` deixa de ser str.

```
from pydantic import BaseModel

class Evento(BaseModel):
    nome: str
    data: str
    local: str
    participantes: ____ # ← que tipo? volta como "12" ou como 12?

agente = Agent(
    name="Extrator de Eventos",
    instructions="Extraia as informações do evento a partir do texto.",
    ____=Evento, # ← o que força a saída nesse formato?
)
evento = Runner.run_sync(agente, texto).final_output # NÃO é mais str
```

4.2 · SAÍDA ESTRUTURADA (OUTPUT_TYPE)

Desafio · Ocorrência: modelos aninhados + lista

DESAFIO

A partir de um relato livre, gere um objeto `Ocorrencia` com tipo, data, local, resumo e uma **lista** de envolvidos, onde cada envolvido é ele mesmo um modelo (`Envolvido` com nome e papel). O modelo define o papel de cada pessoa (vítima / suspeito / testemunha). Percorra a lista no final.

 Solução revelada em aula com a senha da turma

4.3 · SAÍDA ESTRUTURADA (OUTPUT_TYPE)

Desafio · Triagem: Literal (valores fechados) + bool

DESAFIO

Modelo `Triagem` para uma ouvidoria: `categoria` e `prioridade` só aceitam valores de uma lista fechada (`Literal[...]`), mais um `bool urgente` e um `resumo`. O SDK rejeita qualquer valor fora da lista. Mostre que o `bool` volta como bool de verdade (dá para usar em `if t.urgente:`).

 Solução revelada em aula com a senha da turma

Sessão (memória de conversa)

memória — o prontuário do atendimento

EXEMPLOS DE APLICAÇÃO

- Atendimento que coleta aos poucos: o cidadão dá tipo, local e data em turnos separados; a sessão junta tudo e o agente consolida no fim.
- Um caso, uma sessão: o contexto do caso A não aparece no caso B — o isolamento por identificador é requisito de sigilo.
- Chat de chamado: lembra o histórico do atendimento entre mensagens e entre dias (sessão persistida em arquivo).

SAIBA MAIS

Sessão (`Session`) é a memória de conversa entre turnos: sem ela, cada chamada ao Runner começa do zero. A `SQLiteSession` guarda o histórico e devolve para o agente a cada novo turno.

SDK docs — Sessions
<https://openai.github.io/openai-agents-python/sessions/>

SQLiteSession — memória entre turnos

Cada Runner.run é, por padrão, um atendente novo. SQLiteSession(id) guarda o histórico sob um identificador e o reinjeta a cada chamada. Sem db_path é :memory: — mora na RAM.

```
from agents import Agent, Runner, SQLiteSession

sessao = ____("conversa_do_jr")      # ← o objeto de memória (sem db_path = RAM)

r1 = Runner.run_sync(agente, "Qual a capital da França?", ____=sessao)
r2 = Runner.run_sync(agente, "E quantos habitantes ela tem?", ____=sessao)
#                                  ↑ "ela" só funciona com o histórico
```

Desafio · sessão + ferramenta

DESAFIO

Agente com a ferramenta de clima e uma sessão. Pergunte a temperatura de uma cidade, depois de outra e, no 3º turno, qual está mais quente — sem repetir números. A sessão guarda também os resultados das ferramentas, então o 3º turno compara sem nova chamada de API.

 Solução revelada em aula com a senha da turma

Desafio · isolamento entre sessões

DESAFIO

Duas sessões com ids distintos (dois “casos”). Informe um dado na primeira, pergunte na segunda (não deve saber) e confirme na primeira (deve lembrar). O id da sessão é o que impede o vazamento de contexto entre casos — decisão de segurança, não detalhe técnico.

 Solução revelada em aula com a senha da turma

Desafio · coleta progressiva

DESAFIO

Atendimento que coleta aos poucos: o usuário fornece tipo, local e data em turnos separados e, no fim, pede o resumo consolidado. Nenhum turno isolado tem a informação completa — é a sessão que acumula. Sem memória, o último turno não teria o que resumir.

 Solução revelada em aula com a senha da turma

Memória em ARQUIVO (persistência)

DESAFIO

A partir do 5.1, passe `db_path=...` ao `SQLiteSession` para a conversa sobreviver ao fim do programa. Detecte se é a 1ª execução (o arquivo `.db` ainda não existe): na 1ª, registre um dado; nas seguintes, recupere-o. Rode **duas vezes** para provar que o programa fechou entre elas.

 Solução revelada em aula com a senha da turma

Guardrails (travas de segurança)

travas de segurança — detector de metais na entrada + revisão na saída

EXEMPLOS DE APLICAÇÃO

- Trava de LGPD na entrada: bloqueia 'me dá o endereço residencial do servidor João' e explica por que barrou.
- Filtro de PII na saída: se a resposta contiver CPF, telefone ou e-mail, ela é barrada antes de chegar ao cidadão.
- Escopo: o assistente de RH recusa educadamente perguntas fora de RH.
- Anti-injeção: valida o conteúdo que volta de uma tool que leu um documento externo, tratando-o como não-confiável.

SAIBA MAIS

Guardrails são travas que rodam em paralelo ao agente: um `input_guardrail` inspeciona o pedido do usuário antes de gastar tokens com o agente principal; um `output_guardrail` inspeciona a resposta antes de ela sair.

SDK docs — Guardrails
<https://openai.github.io/openai-agents-python/guardrails/>

Input guardrail (Python puro)

Uma trava que roda ANTES do agente. Se detecta algo proibido, dispara o tripwire e a execução para com exceção — o agente principal nem responde. Aqui a checagem é lista de palavras.

```
@input_guardrail
async def bloquear_off_topic(ctx, agent, input) -> GuardrailFunctionOutput:
    texto = input if isinstance(input, str) else str(input)
    violou = any(p in texto.lower() for p in ["receita", "futebol", "fofoca"])
    return GuardrailFunctionOutput(output_info={}, tripwire_triggered=____)
    #                                     ↑ o que dispara o bloqueio?

agente = Agent(name="Tutor de Programação",
               ____=[bloquear_off_topic])      # ← input_guardrails ou output?
# "Como faço um loop?" passa | "receita de bolo?" -> InputGuardrailTripwireTriggered
```

Desafio · input guardrail com LLM (LGPD)

DESAFIO

Um agente “verificador” (com `output_type`) classifica se a mensagem pede dados pessoais sensíveis de terceiros (CPF, endereço residencial, telefone). Use-o dentro de `@input_guardrail`: se sensível, dispare o *tripwire*. Ligue a um “Assistente Institucional” e teste com uma dúvida legítima e uma que peça dados de um servidor.

 Solução revelada em aula com a senha da turma

6.3 · GUARDRAILS (TRAVAS DE SEGURANÇA)

Desafio · output guardrail (regex, sem LLM)

DESAFIO

Um `@output_guardrail` que inspeciona a **resposta** do agente e dispara se ela contiver um CPF, usando expressão regular. Sem `async`, sem LLM — regex Python puro, instantâneo e de graça. Exceção correspondente: `OutputGuardrailTripwireTriggered`.

 Solução revelada em aula com a senha da turma

Streaming (resposta em tempo real)

resposta em tempo real — ver o funcionário digitando

EXEMPLOS DE APLICAÇÃO

- Chat no site da PCDF: efeito 'digitando' + aviso visível 'consultando o sistema...' e 'transferindo para o setor X'.
- Painel do atendente: acompanhar em tempo real qual ferramenta o agente chamou e o que voltou — auditoria ao vivo.
- Respostas longas: relatório ou resumo extenso que o servidor começa a ler antes de terminar de gerar.

SAIBA MAIS

Streaming é receber a resposta do agente em pedaços (eventos), à medida que o modelo gera cada token, em vez de esperar a resposta inteira pronta.

`Runner.run_streamed` devolve um iterador assíncrono de eventos.

SDK docs — Streaming
<https://openai.github.io/openai-agents-python/streaming/>

Tokens, um a um

Runner.run_streamed(...) não leva await — já devolve o objeto de stream. stream.stream_events() é um gerador assíncrono (async def + async for + asyncio.run). Filtramos os eventos de texto cru.

```
import asyncio
from openai.types.responses import ResponseTextDeltaEvent

async def main():
    stream = Runner.run_streamed(agente, "Conte uma micro-história.") # SEM await
    async for event in stream.stream_events():
        if (event.type == "text" and
            isinstance(event.data, ResponseTextDeltaEvent)):
            print(event.data.delta, end="", flush=True)

asyncio.run(main()) # stream_events() é async -> async def + async for + asyncio.run
```

7.2 · STREAMING (RESPOSTA EM TEMPO REAL)

Desafio · streaming + ferramenta (ciclo de vida)

DESAFIO

Além dos tokens do texto, trate `run_item_stream_event`: imprima quando a ferramenta é chamada (`name == "tool_called"`) e quando o resultado chega (`name == "tool_output"`, em `event.item.output`). A turma deve ver, **em ordem**: a ferramenta acionada → o dado voltando → a resposta sendo digitada.

 Solução revelada em aula com a senha da turma

7.3 · STREAMING (RESPOSTA EM TEMPO REAL)

Desafio · streaming + handoff (o momento da transferência)

DESAFIO

Reaproveite um triador (seção 3) e trate `agent_updated_stream_event`: quando o agente ativo mudar, imprima quem assumiu (`event.new_agent.name`) **antes** dos tokens. Torna visível o instante exato do handoff.

 Solução revelada em aula com a senha da turma

Integrador

tudo junto — o atendimento completo

EXEMPLOS DE APLICAÇÃO

- Assistente de ouvidoria completo: responde no tom certo (persona), consulta sistemas (tool), lembra a conversa (sessão), barra dado sensível (guardrail) e entrega a manifestação já estruturada (output_type).
- Balcão virtual de RH: a mesma combinação para dúvidas + consulta ao ponto e férias + registro de uma solicitação.
- Apoio a plantão: consulta de dados cadastrais e de localização, com trava de sigilo e histórico por ocorrência.

SAIBA MAIS

Não é uma capacidade nova do SDK, e sim a **combinação** das anteriores num único agente: tools, handoff, saída estruturada e memória de sessão juntos — o atendimento completo.

SDK docs — Orquestrando vários agentes
https://openai.github.io/openai-agents-python/multi_agent/

Ferramenta + sessão + guardrail + output_type

O atendimento completo num agente só. `get_temperatura` (ferramenta), `guardrail_lgpd` + `ChecagemLGPD` (guardrail + `output_type`), `SQLiteSession` (sessão).

```
assistente = Agent(  
    name="Assistente de Atendimento",  
    instructions="Ajude o cidadão, em português. Use a ferramenta de clima.",  
    tools=[get_temperatura],                # seção 2  
    input_guardrails=[guardrail_lgpd],       # seção 6 (+ output_type na seção 4)  
)  
sessao = SQLiteSession("atendimento_integrador")    # seção 5  
  
# turno 1 -> ferramenta | turno 2 -> memória | turno 3 -> guardrail barra  
# obs: guardrail_lgpd é async e usa `await Runner.run(...)` – run_sync AQUI quebra
```

Limites, segurança e produção

produção — a conversa que decide se o protótipo vira sistema real

EXEMPLOS DE APLICAÇÃO

- Golden set: montar 20 perguntas reais de ouvidoria com a resposta esperada e rodar a cada mudança de prompt ou de modelo.
- Auditoria: guardar o RunResult de cada atendimento para reconstruir o que o agente fez, se questionado.
- Ação com aprovação: se o agente for registrar a ocorrência (não só redigir), um servidor confirma antes de gravar.

SAIBA MAIS

Não é sobre uma classe do SDK, e sim sobre o que separa um protótipo de aula de um sistema em produção: **tracing**/observabilidade, custo e limites de uso, tratamento de erro, e onde os guardrails de fato importam quando o dado é sensível (LGPD).

SDK docs — Tracing (observabilidade)
<https://openai.github.io/openai-agents-python/tracing/>

O loop do agente e o max_turns

Um turno = o modelo decide responder OU chamar ferramenta → se chamou, o Runner executa e devolve → decide de novo.
Runner.run(..., max_turns=N), padrão 10. Estourou → MaxTurnsExceeded.

Prompt injection indireta (pelo conteúdo das ferramentas)

A Seção 6 protege contra o que o cidadão digita. O ataque também vem de dentro: uma ferramenta lê um PDF apreendido / e-mail / página com "IGNORE suas instruções...". Para o modelo, dado e instrução chegam do mesmo jeito — como texto.

Quando NÃO usar um agente

Consulta determinística (validar CPF, buscar CEP no banco, somar) → código comum: mais rápido, de graça, testável, sempre igual. Agente só quando é preciso entender linguagem ambígua ou decidir rota sem regra fixa.

Ação consequente exige aprovação humana

Todos os exemplos do curso só leem. Quando uma ferramenta escreve — registra ocorrência, dispara ofício, altera cadastro, marca alguém — o erro tem efeito direto. O padrão: a ferramenta propõe → um humano aprova → só então roda.

Mesmo funcionário, capacidades somadas

O roteamento e as decisões são feitos em linguagem natural — o código nosso é só a "cola" (ferramentas, travas, memória). Isso é poderoso e exige revisão humana onde a saída vira registro oficial.

Antes de pôr um agente em produção

- Testar — golden set: casos com resposta aceitável, rodados a cada mudança de prompt/modelo.
- Auditar — o RunResult guarda tudo; é a trilha para reconstruir o que o agente fez.
- Custo e latência — 1 turno com ferramenta $\approx$ 2+ chamadas ao modelo; multiplique pelo volume.
- Model drift — o modelo hospedado muda sem aviso; fixe a versão e re-rode o golden set.

Ética e limites no contexto PCDF

- Saída de agente não é ato oficial — sempre há ratificação humana.
- Viés em classificação (vítima/suspeito, risco) — nunca decisão automática, só apoio revisado.
- Residência do dado — dado sob sigilo não vai para API de terceiro; ambiente controlado ou modelo local.

Minimizar o dado na origem, isolar contexto por caso, e ter trava de entrada e de saída.

Expondo um agente como API (FastAPI)

atender por HTTP — o mesmo funcionário, agora também atende por telefone, não só no balcão

EXEMPLOS DE APLICAÇÃO

- Backend de atendimento: outro sistema da casa (intranet, app, outro agente) conversa com o agente via POST /perguntar, sem reescrever a lógica de IA em cada frontend.
- Integração entre sistemas: um sistema de protocolo chama a API do agente de triagem para classificar um documento, sem se acoplar ao SDK.
- Contrato vivo: o Pydantic valida entrada/saída e o /docs (Swagger) documenta a integração para quem for consumi-la.

SAIBA MAIS

FastAPI não substitui o Agent — é a porta HTTP de entrada. Uma rota recebe uma mensagem, chama `Runner.run_sync(agente, mensagem)` e devolve `resultado.final_output` em JSON. Quem decide usar (ou não) uma tool continua sendo o modelo.

FastAPI — documentação <https://fastapi.tiangolo.com/>

Cliente → FastAPI → Agent → Runner → LLM

- Cliente faz um HTTP POST com {"mensagem": "..."}.
- FastAPI valida o corpo (Pydantic) e chama executar_agente(mensagem).
- Agent (instructions + tools) + Runner.run_sync executam o mesmo ciclo das Seções 1–2.
- O LLM decide sozinho se chama uma tool — a API nunca chama a tool direto.
- A resposta final volta como JSON: {"pergunta":..., "resposta":...}.

provedor.py · agente_clima.py · main.py

- provedor.py — mesma configuração do curso (.env, OPENAI_API_KEY/PROVEDOR).
- agente_clima.py — Agent + @function_tool + Runner.run_sync, encapsulados em executar_agente(mensagem).
- main.py — FastAPI: só recebe, valida e devolve; importa executar_agente e chama.

O agente, dentro de uma função reutilizável

Mesmo Agent + @function_tool da Seção 2 — só que agora dentro de executar_agente(mensagem), para o FastAPI chamar.

```
from agents import Agent, Runner, function_tool
from provedor import configurar
configurar()

@_____ # ← mesmo decorator da Seção 2
def consultar_clima(cidade: str) -> str:
    """Retorna o clima atual de uma cidade.

    Args:
        cidade: nome da cidade a consultar.
    """
    dados = {"São Paulo": "22°C, nublado", "Brasília": "28°C, sol"}
    return dados.get(cidade, "Cidade não encontrada.")

agente = Agent(
    name="Assistente de Clima",
    instructions="Use a ferramenta quando perguntarem sobre o tempo.",
    tools=[consultar_clima],
)

def executar_agente(mensagem: str) -> str:
    resultado = Runner.____(agente, mensagem) # ← mesma chamada da Seção 1
    return resultado._____ # ← só o texto final
```

A rota que expõe o agente

FastAPI só recebe, valida (Pydantic) e devolve. Quem decide o que fazer com a mensagem continua sendo o agente.

```
from fastapi import FastAPI
from pydantic import BaseModel
from agente_clima import executar_agente

app = FastAPI()

class Pergunta(BaseModel):
    mensagem: str

@app.post("/perguntar")
def perguntar(pergunta: Pergunta):
    resposta = executar_agente(pergunta.mensagem)
    return {"pergunta": pergunta.mensagem, "resposta": resposta}

# uvicorn main:app --reload → http://127.0.0.1:8000/docs
```

A API não chama a tool — o agente decide

`main.py` nunca escreve `consultar_clima("Brasília")`. Ele chama `executar_agente(mensagem)`; quem decide se precisa da ferramenta é o modelo, dentro do Runner.

- "Como está o tempo em Brasília?" → o modelo decide chamar `consultar_clima`.
- "O que é inteligência artificial?" → o modelo responde sem tool nenhuma.
- Com duas tools, o modelo pode usar nenhuma, uma ou as duas — no mesmo endpoint.

Desafio: múltiplas tools, e depois RAG e multiagentes

- Desafio 1 — acrescente consultar_cotacao(moeda) ao agente e teste: só clima, só cotação, os dois juntos, nenhum dos dois.
- Desafio 2 — crie POST /clima chamando consultar_clima() direto e compare com POST /perguntar: a mesma pergunta, duas arquiteturas.
- Próximo passo natural: a mesma API pode crescer para consultar uma base de conhecimento (RAG) ou rotear entre agentes especialistas (Supervisor + Agente A/B/C) — sem mudar o contrato HTTP.